

Reflected XSS into a JavaScript string with angle brackets HTML encoded

Summary

A reflected Cross-Site Scripting (XSS) vulnerability exists in the application's search functionality.

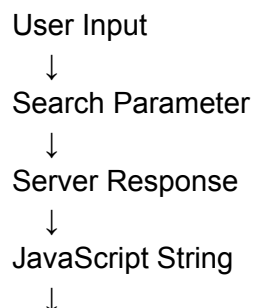
The application reflects user-controlled search input inside a JavaScript string. Although angle brackets are HTML-encoded, the application does not adequately protect the JavaScript string context. As a result, an attacker can terminate the existing string and inject JavaScript.

Vulnerability Classification

Category	Details
Vulnerability	Reflected Cross-Site Scripting
Injection Context	JavaScript string
Input	Search parameter
Payload	'-alert(1)-'
CWE	CWE-79
OWASP	A03:2021 – Injection

Root Cause

The vulnerable data flow can be represented as:



Insufficient Context-Specific Encoding



JavaScript Execution

The application applies HTML encoding to angle brackets, but the search value is still inserted into a JavaScript string.

HTML encoding does not provide sufficient protection for a JavaScript string context. The attacker can therefore use a single quote to escape the existing string.

Exploitation

1. Identify Reflection

A random alphanumeric string was submitted through the search functionality:

test123

The request was intercepted using **Burp Suite** and sent to **Burp Repeater**.

The response was inspected to identify how the supplied value was reflected.

The input was found inside a JavaScript string.

2. Inject Payload

The search value was replaced with:

'-alert(1)-'

This payload terminates the existing JavaScript string and injects an `alert(1)` expression.

3. Verify Execution

The modified request was processed and the resulting URL was copied.

After opening the URL in the browser, an alert dialog displaying `1` was triggered.

This confirmed successful reflected XSS.

Payload Analysis

'-alert(1)-'

The payload works by manipulating the surrounding JavaScript syntax:

- ' terminates the original JavaScript string.
- - helps integrate the injected expression into the existing JavaScript.
- `alert(1)` provides a visible proof of JavaScript execution.
- The final - ' completes the modified expression.

Why HTML Encoding Was Insufficient

The application encoded angle brackets, preventing a straightforward HTML-tag injection approach.

However, the vulnerability was not dependent on HTML parsing. The attacker was able to target the **JavaScript string context** directly.

This highlights an important security principle:

Output encoding must match the context in which untrusted data is inserted.

HTML encoding and JavaScript encoding are not interchangeable.

Impact

Successful exploitation allows attacker-controlled JavaScript to execute within the origin of the vulnerable application.

Depending on the application's functionality, this could potentially allow:

- DOM manipulation.
- Unauthorized actions through the victim's browser.
- Access to information exposed to client-side JavaScript.
- Modification of application content.
- Phishing or UI manipulation.

The actual impact depends on the application's functionality and security controls.

Remediation

The application should prevent untrusted input from being inserted directly into executable JavaScript contexts.

Recommended mitigations:

- Apply context-specific JavaScript output encoding.
- Avoid dynamically constructing JavaScript using user-controlled data.
- Prefer safe data-handling mechanisms over dynamically generated code.
- Validate input according to its intended format.
- Implement a restrictive Content Security Policy (CSP) as defense in depth.

Security Takeaway

This lab demonstrates that **HTML encoding alone does not secure data placed inside JavaScript strings**.

During XSS testing, it is important to identify the exact injection context and determine which characters are interpreted by that context.

The key lesson is to use **context-aware output encoding** and avoid placing untrusted input directly into executable JavaScript.

References

- PortSwigger Web Security Academy — Cross-Site Scripting
- OWASP — Cross-Site Scripting (XSS)
- CWE-79 — Improper Neutralization of Input During Web Page Generation